

Android Activity Lifecycle Lab: Fix the Demo Project

Lab Worksheet

Instructions

Starting from the LifecycleDemo project and handout you've been given, complete the modifications below in order: **Easy** → **Medium** → **Challenge**. Each task tells you what to build, what's broken, and how to verify your fix worked. Record your answers to the "Verify" and "Explain" prompts in the space provided — these will be checked at the end of the lab.

Easy Tasks

Task 1: Fix Data Loss on Rotation

Add an EditText to activity_main.xml. Right now, nothing preserves its contents.

Your job: Implement onSaveInstanceState() to store the text, and restore it in onCreate().

Verify: Type text into the field, rotate the device (Ctrl+F11/F12 in the emulator). The text should still be there.

Explain (write 1–2 sentences): Why does the text disappear without your fix?

Task 2: Fix Duplicate Data Fetch on Rotation

You're given a fetchDataFromServer() method (logs "Fetching...") called from onCreate().

Your job: Guard the call so it only runs on the very first launch, not on every rotation.

Verify: Watch Logcat filtered on your TAG. Rotate the device 3 times — "Fetching..." should log once, not four times.

Explain: What Bundle check distinguishes a fresh launch from a recreation?

Task 3: Add the Missing onRestart() Log

The provided MainActivity logs five callbacks but skips one.

Your job: Add the missing log statement.

Verify: Press Home, then reopen the app from Recents. Confirm the correct three callbacks fire in order, with no onCreate().

Explain: Which three callbacks fired, in what order?

Medium Tasks

Task 4: Fix the Delayed-Callback Crash

You're given a button that starts a 5-second delayed task (Handler.postDelayed) which updates a TextView when it completes.

Your job: Reproduce the crash by tapping the button and rotating the device before the 5 seconds pass. Then fix it using a lifecycle-state check before touching the view.

Verify: Repeat the rotate-before-delay sequence. The app should no longer crash.

Explain: Which two methods did you check, and what does each one tell you?

Task 5: Fix Simulated Battery Drain

You're given a fake "sensor" — a Handler that logs "Sensor reading..." every second, currently started in onCreate() and never stopped.

Your job: Move the start into the correct callback and add a matching stop in its pair.

Verify: Press Home to background the app. Logcat should stop showing sensor readings. Reopen the app — readings should resume.

Explain: Which two callbacks did you use, and why must they be a matched pair?

Task 6: Fix a Memory Leak

You're given a class with a `Handler` (or static field) that implicitly holds a reference to the `Activity`.

Your job: Identify why rotating repeatedly keeps old `Activity` instances alive (use the Android Studio Profiler, or add a log in `onDestroy()` and count instances). Fix it using a static inner class with a `WeakReference`, or by clearing the reference in `onDestroy()`.

Verify: Rotate the device 5 times, then check the Profiler's memory view or your instance-count log — old instances should be freed, not accumulating.

Explain: What was holding the reference, and why did the garbage collector skip it?

Challenge Tasks

Task 7: Fix Real Progress Loss (Process Death)

Build a 3-step wizard in the layout (`currentStep` 0–2, changing which section is visible, e.g. with three stacked `LinearLayout`s toggled by visibility).

Your job: Enable Developer Options → “Don't keep activities.” Step through the wizard, background the app, and confirm the app restarts from step 0. Fix it with `onSaveInstanceState()/restore`.

Verify: With “Don't keep activities” still on, repeat the test — the wizard should resume from the correct step.

Explain: Why doesn't a fix that only handles rotation necessarily survive this test?

Task 8: Migrate to a ViewModel

Instead of the manual guard from Task 2, move the fetched data into a `ViewModel` so it survives rotation without a `savedInstanceState` check at all.

Verify: Rotate the device — no re-fetch should occur, and no manual `Bundle` guard should be needed in `onCreate()`.

Explain: What does the `ViewModel` survive that a plain field does not, and what does it *not* survive that `onSaveInstanceState()` does?

Task 9: Pause-Aware Timer

Add a countdown timer that should keep “counting down” while backgrounded and show the correct remaining time when the app returns — not just resume counting from where the UI froze.

Your job: Store a start timestamp (not a live-decrementing variable) so elapsed time can be recalculated on `onResume()`. Persist it through `onSaveInstanceState()` so it survives process death too.

Verify: Start the timer, background the app for 10+ seconds, reopen it — the displayed time should reflect the real elapsed time, not be frozen or reset.

Explain: Why does storing a timestamp work better here than storing a live counter?

Wrap-Up

For each task you completed, note which lifecycle callback(s) were the key to the fix:

Task	Callback(s) used
1. Data loss on rotation	
2. Duplicate fetch	
3. Missing <code>onRestart()</code>	
4. Delayed-callback crash	
5. Battery drain	
6. Memory leak	
7. Progress loss (process death)	
8. ViewModel migration	
9. Pause-aware timer	